Windows Credential Theft Protection — Credential Guard, LSASS Protection & Windows LAPS (Intune)

Implementation Solution Guide

Version 1.0.0 · 2026-07-29

Contents

Executive Summary — Windows Credential Theft Protection (Credential Guard, LSASS Protection & Windows LAPS) 5

1. Solution Overview — Windows Credential Theft Protection (Credential Guard, LSASS Protection & Windows LAPS) 6

What this solution is . . . 6

The attack this stops (why each control exists) . . . 7

How the pieces work . . . 7

Benefits . . . 8

Limitations (be honest with buyers) . . . 8

The standard deployment model . . . 9

2. Requirements — Windows Credential Theft Protection 9

R1. Windows 10/11 devices enrolled in Intune . . . 9

R2. Hardware readiness for Credential Guard — TPM 2.0, UEFI, Secure Boot, virtualization . . . 10

R3. Supported, patched Windows build . . . 10

R4. Driver / application VBS-compatibility awareness . . . 10

R5. Administrative roles for Endpoint security and configuration . . . 10

R6. Pilot group and a capable-device group . . . 10

R7. Windows LAPS availability (build + Entra tenant feature) . . . 11

R8. LAPS password-retrieval permissions . . . 11

R9. Change-management awareness of the UEFI lock and rollback . . . 11

R10. Pilot user/device and a test remote host (for delegation testing) . . . 11

Global Requirements — Intune Solutions (category-level) 11

G1. Microsoft Entra ID tenant . . . 12

G2. Microsoft Intune subscription . . . 12

G3. Administrative roles . . . 12

G4. Device enrollment in Intune . . . 12

G5. Pilot security group . . . 12

G6. Supported Windows baseline . . . 12

G7. Network access to Microsoft cloud endpoints . . . 12

3. Implementation — Windows Credential Theft Protection 13

Phase 1 — Assess (readiness first) . . . 13

Phase 2 — Configure Credential Guard . . . 14

Phase 3 — Configure LSASS protection (audit-first) . . . 15

Phase 4 — Configure WDigest, Remote Credential Guard, and WinRM . . . 15

Phase 5 — Configure Windows LAPS . . . 17

Phase 6 — Pilot, Validate, Expand . . . 17
5. Testing & Validation — Windows Credential Theft Protection 18
Test 1 — Hardware readiness . . . 19
Test 2 — Profiles present and assigned . . . 19
Test 3 — Credential Guard is running (and nothing broke) . . . 19
Test 4 — LSASS protection audited then enforced . . . 19
Test 5 — WDigest keeps no plaintext . . . 19
Test 6 — Remote Credential Guard restricts outbound delegation . . . 19
Test 7 — WinRM stores no RunAs credential . . . 20
Test 8 — Windows LAPS backup and retrieval . . . 20
Test 9 — Coverage report . . . 20
Test 10 — Reversibility / no unintended breakage (negative test) . . . 20
Acceptance gate . . . 20
6. Troubleshooting — Windows Credential Theft Protection 20
Symptom: A profile never appears on the device . . . 21
Symptom: Credential Guard does not start (SecurityServicesRunning is empty) . . . 21
Symptom: A driver or app breaks after enabling Credential Guard / HVCI . . . 21
Symptom: LSASS protection blocks a security/SmartCard plug-in . . . 21
Symptom: WDigest value still shows 1 (or missing) . . . 22
Symptom: Outbound RDP breaks after Remote Credential Guard . . . 22
Symptom: Windows LAPS password is not backed up to Entra . . . 22
Symptom: Helpdesk cannot retrieve the LAPS password . . . 22
Symptom: Works on some devices, not others . . . 23
Symptom: Coverage report shows fewer devices than the group . . . 23
Escalation checklist . . . 23
7. Best Practices — Windows Credential Theft Protection 23
Assess hardware before you enable VBS . . . 23
Roll out lock mode deliberately . . . 24
Audit LSASS protection first — always . . . 24
Keep the low-risk controls simple and enforced . . . 24
Operate Windows LAPS well . . . 24
Deploy pilot-first and reversibly . . . 24
Operate and monitor . . . 25
Change control . . . 25
8. Deliverables Summary — Windows Credential Theft Protection 25
Documentation (this guide) . . . 25

PowerShell scripts (Microsoft Graph) — `CREDGUARD-Scripts.zip` 26
Hero image 26
How it's delivered 26
Appendix A — PowerShell Scripts 28

Executive Summary — Windows Credential Theft Protection (Credential Guard, LSASS Protection & Windows LAPS)

Most credential-theft attacks on Windows do not break cryptography — they **steal secrets that Windows leaves sitting in memory or on disk**. An attacker who lands on one endpoint dumps **LSASS** for cached password hashes and Kerberos tickets, replays them with **pass-the-hash / pass-the-ticket** to move laterally, harvests **plaintext passwords** WDigest still cached, and pivots through **static local administrator passwords** that are identical on every machine. This solution closes those doors on **Windows 10/11** endpoints managed by **Microsoft Intune** — no domain controllers to touch, no third-party agent, deployed pilot-first and reversible.

It is the **credential-theft / pass-the-hash defense** pack. It deploys, in one coherent playbook, the six controls that together take LSASS off the table as an easy source of credentials:

- **Windows Defender Credential Guard (with UEFI lock)** — isolates secrets (NTLM hashes, Kerberos

TGTs) inside **virtualization-based security (VBS)** so even a SYSTEM-level attacker cannot read them from LSASS.

- **LSASS protection / RunAsPPL** — runs LSASS as a **protected process (PPL)** so unsigned code and

common dumping tools can't open its memory. Deployed **audit-first** so incompatible plug-ins surface before enforcement.

- **Disable WDigest (**`UseLogonCredential = 0`**)** — stops Windows from caching reusable **plaintext**

passwords in LSASS.

- **Restrict delegation of credentials / Remote Credential Guard** — keeps credentials from being

delegated to remote servers over outbound RDP, so a compromised jump host can't harvest them.

- **Disallow WinRM from storing RunAs credentials (**`DisableRunAs = 1`**)** — removes a machine-stored

credential an attacker could recover.

- **Windows LAPS** — backs up, rotates, and retrieves a **unique local administrator password per

device** in Entra ID, ending the identical-local-admin-password lateral-movement path.

Why this matters commercially. A typical security assessment flags several of these directly — most commonly **WinRM RunAs credential storage**, **unrestricted credential delegation to remote servers**, and **WDigest plaintext caching**. This package **resolves those ~3 findings outright** and, with Credential Guard and Windows LAPS on top, **materially raises the endpoint's credential-theft posture** rather than just checking a box.

A deliberately scoped, distinct capability. This is **endpoint-side credential-theft hardening** via Intune. Credential Guard requires **TPM 2.0, UEFI, Secure Boot, and virtualization** and can break some legacy drivers/apps under VBS — so the package includes a **readiness/prerequisite assessment** and a **phased rollout** rather than a fleet-wide flip. The **RDP host-side** settings (Restricted Admin on the target server, NLA, etc.) belong to the separate **RDP hardening** solution and are cross-referenced, not duplicated here.

The package delivers the complete, field-tested method — written so a single administrator can follow it end to end:

• **Named, reusable Intune objects** — `Credential Guard - Pilot`, `LSASS Protection (RunAsPPL) - Pilot`,

`WDigest Disabled - Baseline`, `Remote Credential Guard - Pilot`, `WinRM No RunAs Credentials - Baseline`, and `Windows LAPS - Baseline`, all assigned to the `Credential Protection - Pilot` group.

• **Readiness before enforcement** — a TPM/UEFI/Secure Boot/VBS assessment so Credential Guard only lands

on capable devices, and audit-first LSASS protection so nothing breaks silently.

• **Five ready-to-run PowerShell scripts (Microsoft Graph)** — discovery (tenant + device readiness),

validation, coverage reporting (read-only), plus remediation and scope-targeting that **default to a safe `-WhatIf` preview**.

The package follows the standard repeatable shape — **Assess** → **Configure** → **Pilot** → **Validate** → **Expand** — with exact, click-by-click Intune steps, a requirements guide that tells you how to **verify** *and* **fix** each prerequisite, and testing/troubleshooting guides with clear pass/fail gates.

Outcome: LSASS stops being a credential piñata. Secrets are isolated in VBS, LSASS is a protected process, plaintext caching and machine-stored credentials are gone, credential delegation is restricted, and every device has a unique, rotated local-admin password — proven on a pilot before you scale, without a consulting engagement.

1. Solution Overview — Windows Credential Theft Protection (Credential Guard, LSASS Protection & Windows LAPS)

What this solution is

This solution hardens **Windows 10/11** endpoints against **credential theft and pass-the-hash / lateral movement** using **Microsoft Intune** only — no on-premises Group Policy, no third-party agent. It deploys six coordinated controls, each as a **named, reusable Intune profile** assigned to a single pilot group, then expanded in waves:

1. **Windows Defender Credential Guard** (VBS + LSA isolation, **UEFI lock**) — profile

`Credential Guard - Pilot`.

1. **LSASS protection / RunAsPPL** (LSASS as a protected process, **audit-first**) — profile

`LSASS Protection (RunAsPPL) - Pilot`.

1. **Disable WDigest** (`UseLogonCredential = 0`) — profile `WDigest Disabled - Baseline`.
2. **Restrict credential delegation to remote servers / Remote Credential Guard** — profile

`Remote Credential Guard - Pilot`.

1. **Disallow WinRM from storing RunAs credentials** (`DisableRunAs = 1`) — profile

`WinRM No RunAs Credentials - Baseline`.

1. **Windows LAPS** (Entra ID backup, rotation, retrieval of the local-admin password) — profile

`Windows LAPS - Baseline`.

Together they take **LSASS** — the Local Security Authority Subsystem Service, the process that holds Windows credentials in memory — off the table as an easy source of reusable secrets, and remove the static local-admin password that makes one stolen hash a whole-fleet problem.

> ***Scope, stated plainly.*** *This is the* ***endpoint-side credential-theft defense*** *for Intune-managed Windows 10/11 devices. It is* ***not*** *a full CIS/STIG security baseline (see the baseline solutions),* ***not*** *RDP* ***host*** *hardening (Restricted Admin on the target server, NLA, RDP exposure — see the* ***RDP hardening*** *solution; this package covers only the* ***outbound****, client-side credential protection), and* ***not*** *identity-provider or Conditional Access work. Each of those is a separate solution and is cross-referenced, not duplicated.*

The attack this stops (why each control exists)

An intruder who gains local admin / SYSTEM on one Windows endpoint typically:

• **Dumps LSASS memory** (Mimikatz, `procdump`, `comsvcs.dll`, etc.) to read **NTLM hashes** and

Kerberos tickets, then replays them (**pass-the-hash / pass-the-ticket**) against other systems. → **Credential Guard** moves those secrets into a **VBS-isolated** LSA process (LSAIso) the OS itself cannot read; **RunAsPPL** stops the dumper from opening LSASS in the first place.

• **Reads cached plaintext passwords** WDigest left in LSASS. → **WDigest disabled**

(`UseLogonCredential = 0`) means there is no plaintext to read.

• **Harvests credentials delegated to a jump/RDP host** when an admin connects outbound. → **Remote

Credential Guard / restricted delegation** keeps the credential on the originating device.

• **Recovers a machine-stored WinRM RunAs credential.** → **WinRM** `DisableRunAs = 1` removes it.

• **Reuses the identical local Administrator password** found on the first box to log into every other

box. → **Windows LAPS** gives each device a **unique, rotated** local-admin password escrowed in Entra ID.

How the pieces work

Credential Guard (VBS + LSA isolation)

Credential Guard uses **virtualization-based security (VBS)** to run a trustlet (**LSAIso**) inside a hardware-isolated environment. Derived credentials (NTLM hashes, Kerberos TGTs) live there instead of in normal LSASS memory, so even code running as SYSTEM cannot extract them. It requires **TPM 2.0, UEFI firmware, Secure Boot, and CPU virtualization** and is enabled via Intune's **Device Guard** settings (or the Endpoint security **Account protection** profile). The **UEFI lock** persists the setting in firmware so it cannot be silently disabled by malware. On **Windows 11 22H2 and later**, Credential Guard is **on by default** on eligible, newly-installed devices — this solution makes it **explicit, enforced, and reported** across the fleet.

LSASS protection (RunAsPPL / PPL)

Running LSASS as a **Protected Process Light (PPL)** (`RunAsPPL = 1`) means only Microsoft-signed code can open its process memory, blocking most credential-dumping tools. Because a mis-signed security plug-in or SmartCard/driver can be incompatible, this is deployed **audit-first**: enable, watch for the "would-block" events, remediate the offender, then enforce (with UEFI lock).

WDigest, Remote Credential Guard, WinRM

These are small, well-understood registry/ADMX settings delivered through the Settings Catalog: `UseLogonCredential = 0` (no plaintext caching), **Restrict delegation of credentials to remote servers** → **Require Remote Credential Guard** on outbound RDP, and **Disallow WinRM from storing RunAs credentials**. Low-risk and reversible.

Windows LAPS

Windows LAPS (built into current Windows, no legacy MSI) rotates the local administrator password on a schedule and **backs it up to Entra ID**, where an authorized admin retrieves it on demand. It replaces the anti-pattern of a shared, static local-admin password baked into the image. Deployed through Endpoint security **Account protection**.

Benefits

• **Kills the pass-the-hash chain at the endpoint** — VBS isolation + PPL + no plaintext means the first compromised box no longer yields reusable fleet-wide credentials.

• **Resolves real assessment findings** — WinRM RunAs storage, unrestricted credential delegation, and WDigest plaintext caching are common audit hits this package closes directly.

• **Unique local-admin password per device** — LAPS removes the single most common lateral-movement shortcut with Entra-backed retrieval and rotation.

• **Readiness-gated and reversible** — Credential Guard lands only on TPM/UEFI/Secure-Boot-capable devices; every control rolls back by unassigning its profile (subject to the UEFI-lock note below).

• **Intune-native** — configuration profiles and Endpoint security policies only; no GPO, no extra agent.

Limitations (be honest with buyers)

• **Credential Guard has hardware prerequisites.** TPM 2.0, UEFI, Secure Boot, and virtualization are required; older or misconfigured firmware won't qualify. The readiness assessment (and `CREDGUARD-Discovery.ps1 -LocalReadiness`) finds these before you deploy.

• **VBS can break some legacy drivers/apps.** Kernel-mode drivers that aren't HVCI-compatible, older VPN or endpoint agents, or nested-virtualization scenarios may misbehave — hence **pilot-first**.

• **UEFI lock is deliberately sticky.** With the UEFI lock set, disabling Credential Guard is not a

simple profile-unassign — it requires the documented removal steps (and often physical/console access). This is a security feature; the guide covers the trade-off and the without-lock option for pilots.

- **Windows LAPS needs a capable Windows build and the Entra tenant switch enabled** (April 11 2023

cumulative update or later; Entra local-admin-password feature turned on).

- **This is endpoint hardening, not the whole story.** Domain-side Protected Users, tiered

administration, and RDP **host** hardening are complementary and out of scope here.

- **Verify identifiers/settings in your tenant.** Setting names and defaults evolve across Windows and

Intune releases — the guide flags where to confirm rather than assume.

The standard deployment model

This solution uses **one** repeatable model across our catalog:

> ***Assess*** *device hardware readiness and the current state of each control →* ***Configure*** *the six named Intune profiles (Credential Guard + LSASS PPL audit-first + WDigest + Remote Credential Guard + WinRM + LAPS) →* ***Pilot*** *against* `Credential Protection - Pilot` *→* ***Validate*** *with the scripts and on-device checks →* ***Expand*** *in waves.*

Every step follows the repeatable pattern: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.**

2. Requirements — Windows Credential Theft Protection

Confirm **every** requirement below before you begin. Each entry follows the same pattern: **Description · Why required · How to verify · How to implement if missing.** Category-wide prerequisites (tenant, Intune licensing, admin roles, pilot group, network) are covered once in the **Global Requirements — Intune Solutions** document; the items here are specific to deploying Credential Guard, LSASS protection, WDigest/WinRM/credential-delegation hardening, and Windows LAPS to corporate Windows 10/11 endpoints.

> ***Scope reminder.*** *These requirements are for* ***endpoint-side credential-theft hardening*** *on* ***enrolled Windows 10/11*** *devices. They deliberately do* ***not*** *include RDP* ***host*** *hardening, full CIS/STIG baselines, or domain-controller changes — those are separate solutions.*

R1. Windows 10/11 devices enrolled in Intune

Description: The target endpoints are Intune-enrolled/managed Windows 10 (21H2+) or Windows 11 devices (Entra-joined or hybrid-joined). **Why:** All six controls are delivered as Intune configuration/Endpoint-security policies; they only reach enrolled devices. **Verify:** Intune admin center (`https://intune.microsoft.com`) → **Devices** → **Windows** → **Windows devices**; the device is listed and **managed/compliant**. **Implement:** Enable **Automatic Enrollment** (Entra → Mobility (MDM) → Microsoft Intune) and Entra-join the device, or enroll via Autopilot; confirm it appears under **Devices** → **Windows**.

R2. Hardware readiness for Credential Guard — TPM 2.0, UEFI, Secure Boot, virtualization

Description: Devices targeted for Credential Guard have **TPM 2.0**, **UEFI firmware** (not legacy BIOS), **Secure Boot enabled**, and **CPU virtualization** (Intel VT-x/AMD-V + SLAT) available. **Why:** Credential Guard runs inside **virtualization-based security (VBS)**; without these the VBS/LSA isolation cannot start and the policy has no effect (or errors). **Verify:** On a candidate device run `CREDGUARD-Discovery.ps1 -LocalReadiness`, or check manually: `Get-Tpm` (TpmPresent/TpmReady **True**), `Confirm-SecureBootUEFI` (**True**), `msinfo32` → **System Summary** (BIOS Mode = **UEFI**, Secure Boot State = **On**), and **Virtualization-based security** state. **Implement:** Enable **TPM**, **UEFI/Secure Boot**, and **virtualization (VT-x/AMD-V)** in firmware (often OEM-manageable); replace/upgrade devices that lack TPM 2.0. Use the readiness output to build the **capable-devices** group before assigning `Credential Guard - Pilot`.

R3. Supported, patched Windows build

Description: A current, supported Windows 10 (21H2+) or Windows 11 release with recent cumulative updates. **Why:** VBS/Credential Guard, LSA protection defaults, and Windows LAPS availability depend on the OS build; recent releases fix known compatibility issues. **Verify:** `winver` on the device, or Intune → device → **Hardware** → **Operating system version**. **Implement:** Deploy **Update rings** / **Feature updates**, or upgrade to Windows 11. For LAPS specifically see R7.

R4. Driver / application VBS-compatibility awareness

Description: Awareness that some **kernel-mode drivers** and legacy agents (older VPN, endpoint security, virtualization tools) may be incompatible with VBS/HVCI or with LSASS running as a protected process. **Why:** Enabling Credential Guard or RunAsPPL on an incompatible device can cause driver failures or blocked LSA plug-ins — this is why the rollout is pilot- and audit-first. **Verify:** During the pilot, review **Event Viewer** for VBS/HVCI driver-compatibility events and, for LSA protection, the LSA "would-block" events (Testing, Test 3 and Test 4). **Implement:** Update or replace incompatible drivers/agents before expanding; keep the device in the pilot group until clean.

R5. Administrative roles for Endpoint security and configuration

Description: The operator holds **Intune Administrator** (or Endpoint Security Manager) rights, and LAPS retrieval will require the appropriate Entra role/permission (R8). **Why:** Creating Device Guard / Settings Catalog profiles and **Account protection** (Credential Guard, LAPS) policies requires elevated Intune rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide switches such as enabling the Entra LAPS feature (R7).

R6. Pilot group and a capable-device group

Description: An Entra security group `Credential Protection - Pilot` for the initial rollout, plus (recommended) a group scoped to **Credential-Guard-capable** devices (R2) for the VBS controls. **Why:** Every control is piloted before fleet rollout; Credential Guard should target only capable hardware to avoid errors on unsupported devices. **Verify:** Entra → **Groups**; `Credential Protection - Pilot` exists and contains the pilot devices/users. **Implement:** Entra → **Groups** → **New group** → **Security**; add pilot members. Build the capable-device group from the readiness assessment (R2) — assigned membership, or a dynamic rule once you have a device-attribute signal.

R7. Windows LAPS availability (build + Entra tenant feature)

Description: The modern **Windows LAPS** (built into Windows) is available on the device, and the **Entra ID local administrator password (LAPS)** feature is enabled in the tenant. **Why:** Windows LAPS with Entra backup requires the **April 11 2023** cumulative update (or later) on a supported build, and the tenant-level switch must be **on** or Entra backup fails. **Verify:** Device: `Get-Command Set-LapsADPassword`/`Get-LapsDiagnostics` exist and the **Microsoft-Windows-LAPS/Operational** log is present. Tenant: Entra → **Devices** → **Device settings** (or **All devices** → **Device settings**) → **Enable Microsoft Entra Local Administrator Password Solution (LAPS)** = **Yes**. **Implement:** Patch devices to the required build; in Entra set the LAPS device setting to **Yes** (Global Administrator or the appropriate role). Do **not** run the legacy on-prem LAPS MSI alongside — use built-in Windows LAPS.

R8. LAPS password-retrieval permissions

Description: The admins who will retrieve local-admin passwords have the right to read them from Entra. **Why:** Retrieval is gated by an Entra role/permission (`deviceLocalCredentials`); without it the password is stored but not readable by the helpdesk. **Verify:** In Entra → **Devices** → **(a device)** → **Local administrator password recovery**, an authorized admin can show the current password. **Implement:** Grant a role that includes `microsoft.directory/deviceLocalCredentials/password/read` (e.g. **Cloud Device Administrator** / **Intune Administrator**, or a custom role); document who may retrieve.

R9. Change-management awareness of the UEFI lock and rollback

Description: Awareness that the **UEFI lock** on Credential Guard (and optionally LSA protection) makes those controls **sticky** — they are not removed by simply unassigning the profile. **Why:** Buyers must plan rollback: removing UEFI-locked VBS requires the documented removal procedure (and often console/physical access), by design. **Verify:** Decide per phase whether to use **Enabled with UEFI lock** (production) or **Enabled without lock** (pilots, easy rollback). **Implement:** Use **without lock** during the pilot; switch to **with lock** for production waves once compatibility is proven (Implementation Step 5 and Best Practices).

R10. Pilot user/device and a test remote host (for delegation testing)

Description: At least one pilot device on capable hardware, plus a second Windows host to RDP **to** when validating Remote Credential Guard. **Why:** The credential-delegation control is validated by making an outbound RDP connection and confirming credentials are not delegated. **Verify:** The pilot device is enrolled (R1), capable (R2), in the pilot group (R6); a reachable RDP target exists. **Implement:** Add the pilot device to `Credential Protection - Pilot`; identify an RDP target host (its **host-side** hardening is the separate RDP solution).

Global Requirements — Intune Solutions (category-level)

These requirements apply to **every** solution in the **Intune Solutions** category. They are published once at the category level so individual listings stay small. Confirm these before deploying *any* Intune solution. Each entry: **Description · Why · Verify · Implement-if-missing.**

G1. Microsoft Entra ID tenant

Description: An active Microsoft Entra ID (Azure AD) tenant. **Why:** It is the identity backbone for every cloud and Intune-managed resource. **Verify:** Sign in to `https://entra.microsoft.com` as an administrator. **Implement:** Create a Microsoft 365/Azure subscription, which provisions a tenant.

G2. Microsoft Intune subscription

Description: An Intune-bearing license plan. **Why:** Intune delivers all configuration, compliance, and security policies. **Verify:** `https://intune.microsoft.com` loads; **Tenant administration** → **Tenant status** shows active. **Implement:** Assign Microsoft 365 E3/E5, Business Premium, or standalone Intune Plan 1.

G3. Administrative roles

Description: Appropriate Entra/Intune admin roles for the operator. **Why:** Creating policies and enrollment settings requires elevated rights. **Verify:** Entra → **Roles and administrators**; confirm **Intune Administrator**. **Implement:** Assign **Intune Administrator**; reserve Global Administrator for tenant-wide changes.

G4. Device enrollment in Intune

Description: Targeted Windows devices are enrolled and manageable. **Why:** Policies only reach enrolled devices. **Verify:** Intune → **Devices** → **All devices**; the device appears and is **compliant**. **Implement:** Enable **Automatic Enrollment** and join devices to Entra ID.

G5. Pilot security group

Description: A Microsoft Entra security group for pilot/targeted users or devices. **Why:** Every solution is piloted before fleet rollout. **Verify:** Entra → **Groups**; the pilot group exists. **Implement:** Entra → **Groups** → **New group** → **Security** → add members.

G6. Supported Windows baseline

Description: Windows 10 21H2+ or Windows 11 with current updates. **Why:** Modern Intune features assume a patched, supported OS. **Verify:** `winver`, or Intune → device → **Hardware**. **Implement:** Deploy **Update rings** or upgrade to Windows 11.

G7. Network access to Microsoft cloud endpoints

Description: Outbound HTTPS to Microsoft 365, Entra, and Intune service URLs. **Why:** Enrollment, policy sync, and authentication depend on these endpoints. **Verify:** From a device, reach `https://login.microsoftonline.com` and `https://enrollment.manage.microsoft.com`. **Implement:** Allow Microsoft's published Office 365/Intune URL & IP ranges through proxy/firewall.

The standard deployment model (all solutions): Assess → Configure → Pilot → Validate → Expand.

3. Implementation — Windows Credential Theft Protection

Follow these steps **in order**. The *why* for each capability lives in the Solution Overview; the steps below are deliberately short and exact so you can **click to implement**. Throughout, the repeatable pattern applies: **Assess** → **Configure** → **Pilot** → **Validate** → **Expand.** Do everything against the `Credential Protection - Pilot` group first.

All admin-center steps use the Microsoft Intune admin center at `https://intune.microsoft.com` and the Microsoft Entra admin center at `https://entra.microsoft.com` unless noted.

> ***This hardens the endpoint against credential theft only.*** *It does not harden RDP* ***hosts*** *(separate solution), is not a full CIS/STIG baseline, and makes no domain-controller changes. Deploy the VBS controls (Credential Guard, LSASS PPL) to* ***capable hardware*** *and* ***pilot-first****.*

Objects you will create (named once, reused throughout testing and troubleshooting):

Object	Type	Purpose
`Credential Protection - Pilot`	Entra security group	Pilot assignment target
`Credential Guard - Pilot`	Settings Catalog (Device Guard)	VBS + Credential Guard (UEFI lock)
`LSASS Protection (RunAsPPL) - Pilot`	Settings Catalog (Local Security Authority)	LSASS as protected process, audit-first
`WDigest Disabled - Baseline`	Settings Catalog	`UseLogonCredential = 0`
`Remote Credential Guard - Pilot`	Settings Catalog (Admin Templates → Credentials Delegation)	Restrict outbound credential delegation
`WinRM No RunAs Credentials - Baseline`	Settings Catalog (Admin Templates → WinRM Service)	`DisableRunAs = 1`
`Windows LAPS - Baseline`	Endpoint security → Account protection	Entra-backed local-admin password

Phase 1 — Assess (readiness first)

Step 1. Confirm enrollment and hardware readiness

1. Go to **Devices** → **Windows** → **Windows devices** and confirm the pilot device is **managed** and **compliant**.

1. On a candidate device, run the readiness check:

`powershell -ExecutionPolicy Bypass -File .\CREDGUARD-Discovery.ps1 -LocalReadiness` Confirm **TPM 2.0 present/ready**, **UEFI + Secure Boot On**, and **virtualization available**. (Or check manually per Requirements R2.)

1. Record which devices are **Credential-Guard-capable**; put those in a capable-device group (R6). The

VBS controls (`Credential Guard - Pilot`, `LSASS Protection (RunAsPPL) - Pilot`) target that group; the low-risk controls (WDigest, WinRM) can target all pilot devices.

Step 2. Create the pilot group

1. Entra → **Groups** → **New group**. **Group type** = **Security**; **Group name** = `Credential Protection - Pilot`. **Create**.
1. Add your pilot device(s)/user(s) as members.

Phase 2 — Configure Credential Guard

Step 3. Create the Credential Guard profile (Settings Catalog → Device Guard)

1. Go to **Devices** → **Configuration** → **Create** → **New policy**.
2. **Platform** = **Windows 10 and later**; **Profile type** = **Settings catalog**; **Create**.
3. **Basics** → **Name:** `Credential Guard - Pilot`. Description, e.g. *VBS + Windows Defender Credential Guard (UEFI lock) for credential-theft protection.* **Next**.
1. On **Configuration settings**, click **+ Add settings**. In the settings picker search **Device Guard **and select the** Device Guard** category. Add:

• **Turn On Virtualization Based Security** → **Enabled**.

• **Select Platform Security Level** → **Secure Boot** (or **Secure Boot and DMA Protection** on devices with kernel-DMA protection).

• **Credential Guard Configuration** → **Enabled with UEFI lock** (production) *or* **Enabled without lock** (pilot / easy rollback — see Step 5).

• *(Optional, recommended)* **Virtualization Based Protection of Code Integrity (HVCI)** → **Enabled with UEFI lock** — memory-integrity hardening that complements Credential Guard.

1. **Next** through **Scope tags**.

Step 4. Assign the Credential Guard profile

1. **Assignments** → **Add groups** → add `Credential Protection - Pilot` (or the capable-device group). **Next**.
1. **Review + create** → **Create.**

Step 5. Choose the lock mode deliberately (pilot vs production)

1. For the **pilot**, prefer **Enabled without lock** so you can disable Credential Guard by unassigning the profile if a driver/app breaks.
1. For **production waves**, switch the same setting to **Enabled with UEFI lock** so malware cannot

silently disable it. Note (R9): with the UEFI lock, removal requires the documented VBS-removal procedure, not just a profile unassign.

> *Prefer automation?* `CREDGUARD-Remediation.ps1` *can create this profile and assign it (defaults to* ***-WhatIf*** *— preview first, then re-run to apply), and* `CREDGUARD-Discovery.ps1` *confirms it is present and correctly configured.*

Phase 3 — Configure LSASS protection (audit-first)

Step 6. Create the LSASS protection profile (RunAsPPL)

1. **Devices** → **Configuration** → **Create** → **New policy** → **Windows 10 and later** → **Settings catalog** →

Create.

1. **Name:** `LSASS Protection (RunAsPPL) - Pilot`. **Next**.

2. **+ Add settings** → search **Local Security Authority** and select the **Local Security Authority

(LSA) **category. Add** Configures LSASS to run as a protected process **and set it to** Enabled without UEFI lock **for the pilot (so you can back it out if an LSA plug-in is incompatible). Switch to** Enabled with UEFI lock** for production once clean.

• *(If your tenant does not surface this setting, deliver the equivalent registry value

`HKLM\SYSTEM\CurrentControlSet\Control\Lsa\RunAsPPL = 1` via a **Custom** OMA-URI / a proactive remediation. **Verify the exact setting name in your tenant** — LSA-protection surfacing varies by Windows/Intune release.)*

1. **Assign** to `Credential Protection - Pilot`. **Review + create** → **Create.**

Step 7. Audit before enforcing

1. Leave `LSASS Protection (RunAsPPL) - Pilot` deployed for an observation window (e.g. one to two

weeks) on the pilot.

1. On pilot devices, review **Event Viewer → Applications and Services Logs → Microsoft → Windows →

CodeIntegrity → Operational **(and the LSA operational log) for** "would-block" LSA plug-in/driver **events (commonly** Event ID 3065 **= would-block if enforced,** 3066** = blocked). These name any incompatible module.

1. Update or remove the flagged module(s). Only when the audit window is clean, switch the setting to

Enabled with UEFI lock for production waves. **Verify current event IDs in your build** — they evolve.

Phase 4 — Configure WDigest, Remote Credential Guard, and WinRM

Step 8. Disable WDigest (`WDigest Disabled - Baseline`)

1. **Devices** → **Configuration** → **Create** → **New policy** → **Windows 10 and later** → **Settings catalog** →

Create. **Name:** `WDigest Disabled - Baseline`. **Next**.

1. **+ Add settings** → search **WDigest** / **UseLogonCredential**. Add the setting that maps to `HKLM\SYSTEM\CurrentControlSet\Control\SecurityProviders\WDigest\UseLogonCredential` and set it to **0 / Disabled** (no plaintext caching).

• *(If not surfaced in your tenant's Settings Catalog, deliver `UseLogonCredential = 0` via a **Custom** OMA-URI or proactive remediation. Modern Windows already defaults this to 0 — this profile **enforces and proves** it.)*

1. **Assign** to `Credential Protection - Pilot`. **Review + create** → **Create.**

Step 9. Restrict credential delegation / Remote Credential Guard (`Remote Credential Guard - Pilot`)

1. **Devices** → **Configuration** → **Create** → **New policy** → **Windows 10 and later** → **Settings catalog** →

Create. **Name:** `Remote Credential Guard - Pilot`. **Next**.

1. **+ Add settings** → search **Credentials Delegation** and select **Administrative Templates → System → Credentials Delegation**. **Add** Restrict delegation of credentials to remote servers → Enabled**, and set** Use the following restricted mode → Require Remote Credential Guard **(this protects credentials when the device connects** outbound **over RDP). *(Choose** Restrict Credential Delegation** if you need the strictest mode and your targets support it.)*

1. **Assign** to `Credential Protection - Pilot`. **Review + create** → **Create.**

> ***Host side is a different product.*** *The matching **RDP host** setting — *Remote host allows delegation of non-exportable credentials* (`AllowProtectedCreds = 1`) — plus Restricted Admin mode and NLA on the **target server** are covered by the separate **RDP hardening** solution. This profile is the **client/outbound** half only.*

Step 10. Disallow WinRM RunAs credential storage (`WinRM No RunAs Credentials - Baseline`)

1. **Devices** → **Configuration** → **Create** → **New policy** → **Windows 10 and later** → **Settings catalog** →

Create. **Name:** `WinRM No RunAs Credentials - Baseline`. **Next**.

1. **+ Add settings** → search **Windows Remote Management** and select **Administrative Templates → Windows Components → Windows Remote Management (WinRM) → WinRM Service**. **Add** Disallow WinRM from storing RunAs credentials → Enabled** (sets `HKLM\SOFTWARE\Policies\Microsoft\Windows\WinRM\Service\DisableRunAs = 1`).

1. **Assign** to `Credential Protection - Pilot`. **Review + create** → **Create.**

> *These three low-risk controls may be combined into a single Settings Catalog profile if you prefer fewer objects; the guide keeps them separate so each is named, reported, and rolled back independently.*

Phase 5 — Configure Windows LAPS

Step 11. Enable the Entra LAPS tenant feature (prerequisite)

1. Entra → **Devices** → **Device settings** (or **All devices** → **Device settings**).

2. Set **Enable Microsoft Entra Local Administrator Password Solution (LAPS)** = **Yes**. **Save.**

(Requires the appropriate role — R7.)

Step 12. Create the Windows LAPS policy (Endpoint security → Account protection)

1. Go to **Endpoint security** → **Account protection** → **Create Policy**.

2. **Platform** = **Windows**; **Profile** = **Local admin password solution (Windows LAPS)**. **Create**.

3. **Name:** `Windows LAPS - Baseline`. **Next**.

4. **Configuration settings:**

• **Backup Directory** → **Backup the password to Azure AD (Microsoft Entra ID) only**.

• **Password Age (days)** → e.g. **30**.

• **Administrator Account Name** → leave blank to manage the built-in Administrator, or specify a named local admin account you manage.

• **Password Complexity** → large letters + small letters + numbers + special (strongest).

• **Password Length** → e.g. **20**.

• **Post Authentication Actions** → **Reset the password and logoff the managed account** after the configured grace period.

• **Verify the exact field labels in your tenant** — Account-protection LAPS settings evolve.

1. **Assign** to `Credential Protection - Pilot`. **Review + create** → **Create.**

Step 13. Confirm retrieval works

1. On a pilot device, force a sync (**Settings** → **Accounts** → **Access work or school** → **Info** → **Sync**) and

wait for the policy to apply.

1. In Entra → **Devices** → **(the pilot device)** → **Local administrator password recovery**, confirm an authorized admin can **show** the current password (R8). This proves backup + retrieval end to end.

Phase 6 — Pilot, Validate, Expand

Step 14. Validate with the scripts

1. Run `CREDGUARD-Discovery.ps1` (tenant readiness: all six profiles present and, on a device, `-LocalReadiness` for TPM/UEFI/Secure Boot/VBS).

1. Run `CREDGUARD-Validation.ps1` to confirm each profile is **assigned** to `Credential Protection - Pilot` and pilot devices report **applied/Succeeded**.

1. Run `CREDGUARD-Report.ps1` to produce a coverage report (which Windows devices have the credential-protection controls applied, plus readiness/VBS-running state where available).

Step 15. Confirm on the device

1. Credential Guard running:

`(Get-CimInstance -ClassName Win32_DeviceGuard -Namespace root\Microsoft\Windows\DeviceGuard).SecurityServicesRunning` should contain **1** (Credential Guard); or `msinfo32` → **Virtualization-based security Services Running: Credential Guard**.

1. LSASS protected: registry `HKLM\SYSTEM\CurrentControlSet\Control\Lsa\RunAsPPL = 1`.

2. WDigest: `HKLM\...\SecurityProviders\WDigest\UseLogonCredential = 0`.

3. WinRM: `HKLM\SOFTWARE\Policies\Microsoft\Windows\WinRM\Service\DisableRunAs = 1`.

4. LAPS: **Microsoft-Windows-LAPS/Operational** log shows a successful backup; retrieval works (Step 13).

Step 16. Remediate gaps (safe by default)

1. If a profile is missing or mis-scoped, run `CREDGUARD-Remediation.ps1 -GroupId <capable-or-pilot group> — it defaults to -WhatIf and prints exactly what it would create/assign. Review, then re-run with -WhatIf:$false to apply.

Step 17. Expand in waves

1. Once the pilot passes (Testing gate) and the LSA audit window is clean, switch the VBS controls to **Enabled with UEFI lock** and broaden assignment to the next wave via `CREDGUARD-Set-ScopeTargeting.ps1` (defaults to **-WhatIf**).

1. Keep Credential Guard / LSASS scoped to **capable** devices; let WDigest, WinRM, Remote Credential Guard, and LAPS ride out to all managed Windows devices.

1. Re-validate each wave with `CREDGUARD-Report.ps1` before widening.

5. Testing & Validation — Windows Credential Theft Protection

Validate against a **pilot device** on **capable hardware** before expanding. Each test has an explicit **pass/fail gate**. Use the included PowerShell scripts (Microsoft Graph) for tenant-side checks and the on-device steps for the runtime state.

> ***What "pass" means here:*** *on the pilot device, Credential Guard is **running**, LSASS is a **protected process**, WDigest keeps **no plaintext**, credential delegation is **restricted**, WinRM stores **no RunAs***

*credential, and the device has a **unique, retrievable** LAPS password — with no broken drivers/apps.*

Test 1 — Hardware readiness

Procedure: Run `CREDGUARD-Discovery.ps1 -LocalReadiness` on the candidate device. **Pass:** TPM 2.0 present/ready, UEFI + Secure Boot **On**, virtualization available — device is **Credential-Guard-capable**. **Fail:** Any prerequisite missing → enable in firmware or exclude the device from the VBS controls (R2).

Test 2 — Profiles present and assigned

Procedure: Run `CREDGUARD-Discovery.ps1` then `CREDGUARD-Validation.ps1`. **Pass:** All six profiles (`Credential Guard - Pilot`, `LSASS Protection (RunAsPPL) - Pilot`, `WDigest Disabled - Baseline`, `Remote Credential Guard - Pilot`, `WinRM No RunAs Credentials - Baseline`, `Windows LAPS - Baseline`) exist and are assigned to `Credential Protection - Pilot`; pilot devices report **Succeeded**. **Fail:** Missing/unassigned profile → create/assign (Implementation) or run `CREDGUARD-Remediation.ps1` (`-WhatIf` first).

Test 3 — Credential Guard is running (and nothing broke)

Procedure: On the pilot device run `(Get-CimInstance -ClassName Win32_DeviceGuard -Namespace root\Microsoft\Windows\DeviceGuard).SecurityServicesRunning` (or `msinfo32` → *Virtualization-based security Services Running*). Reboot and confirm normal operation. **Pass:** `SecurityServicesRunning` contains **1** (Credential Guard); the device boots and core apps/drivers work. **Fail:** Credential Guard not running → confirm VBS platform-security level and hardware (Test 1); if a driver fails, review VBS/HVCI driver-compatibility events (R4) and remediate before expanding.

Test 4 — LSASS protection audited then enforced

Procedure: With `LSASS Protection (RunAsPPL) - Pilot` deployed, review **Event Viewer** → **… CodeIntegrity** → **Operational** (and the LSA log) for **would-block** events (e.g. **3065/3066**); then confirm `HKLM\SYSTEM\CurrentControlSet\Control\Lsa\RunAsPPL = 1`. **Pass:** No unresolved would-block events during the audit window; `RunAsPPL = 1`; LSASS runs protected. **Fail:** A plug-in/driver is flagged as incompatible → update/remove it and re-audit before switching to UEFI-lock enforcement (Implementation Step 7).

Test 5 — WDigest keeps no plaintext

Procedure: Check `HKLM\SYSTEM\CurrentControlSet\Control\SecurityProviders\WDigest\UseLogonCredential`. **Pass:** Value = **0** — Windows does not cache reusable plaintext passwords in LSASS. **Fail:** Value **1** or missing → confirm `WDigest Disabled - Baseline` applied; re-sync or deliver via the Custom/remediation fallback (Implementation Step 8).

Test 6 — Remote Credential Guard restricts outbound delegation

Procedure: From the pilot device, RDP **out** to the test host (R10) and confirm the connection uses Remote Credential Guard (no credentials delegated). Confirm the policy value `HKLM\SOFTWARE\Polici`

`es\Microsoft\Windows\CredentialsDelegation\RestrictedRemoteAdministration = 1` with the Remote Credential Guard mode. **Pass:** The outbound RDP session works and the originating credential is **not** delegated to the host. **Fail:** Delegation still occurs, or RDP breaks → verify the mode chosen (Implementation Step 9) and that the **host side** supports it (that is the separate RDP hardening solution, not this profile).

Test 7 — WinRM stores no RunAs credential

Procedure: Check `HKLM\SOFTWARE\Policies\Microsoft\Windows\WinRM\Service\DisableRunAs`. **Pass:** Value = **1** — WinRM will not store a RunAs credential. **Fail:** Missing/0 → confirm `WinRM No RunAs Credentials - Baseline` applied and re-sync.

Test 8 — Windows LAPS backup and retrieval

Procedure: On the pilot device confirm the **Microsoft-Windows-LAPS/Operational** log shows a successful password backup; in Entra → **Devices** → **(device)** → **Local administrator password recovery**, retrieve the current password as an authorized admin (R8). **Pass:** A unique local-admin password is backed up to Entra and retrievable; rotation age matches the policy. **Fail:** No backup / not retrievable → confirm the Entra LAPS tenant feature is **On** (Step 11), the build supports Windows LAPS (R7), and the retriever has the permission (R8).

Test 9 — Coverage report

Procedure: Run `CREDGUARD-Report.ps1`. **Pass:** The report lists the pilot device(s) with the credential-protection controls **applied**; counts match the assigned group. **Fail:** Devices missing → run `CREDGUARD-Remediation.ps1` (`-WhatIf` first) and re-sync stragglers.

Test 10 — Reversibility / no unintended breakage (negative test)

Procedure: On a **without-lock** pilot device, unassign a VBS profile and confirm the control backs out cleanly; confirm no unrelated policy changed. **Pass:** Without-lock controls revert on unassignment; only the six named profiles were introduced. (For **UEFI-locked** production devices, removal follows the documented VBS-removal procedure — R9.) **Fail:** Behavior does not revert as expected for a without-lock device → confirm which lock mode was set and re-check assignment scope.

Acceptance gate

Proceed to **Expand** (and to **UEFI-lock enforcement**) only when Tests 1–9 pass on at least one capable pilot device, the LSASS audit window (Test 4) is clean, Test 10 confirms reversibility for without-lock devices, and a real pilot user confirms day-to-day apps, VPN, RDP, and sign-in behave normally.

6. Troubleshooting — Windows Credential Theft Protection

Work top-down: confirm the **profile is applied**, then the **hardware supports VBS**, then the **runtime state** (Credential Guard running, RunAsPPL set), then **compatibility** (drivers/plug-ins). Most "Credential Guard won't turn on" cases are a missing hardware prerequisite; most "it broke something" cases are an

incompatible kernel driver or LSA plug-in surfaced by skipping the audit window. Each symptom below gives the **likely cause** and the **fix**.

> ***Before you start:*** *confirm which control you are diagnosing. Credential Guard, LSASS PPL, WDigest, Remote Credential Guard, WinRM, and LAPS are independent profiles — isolate to the one profile whose test failed rather than changing all six.*

Symptom: A profile never appears on the device

Likely cause: Device not synced, not assigned, or not enrolled. **Fix:**

1. Confirm the device is in `Credential Protection - Pilot` and the profile is **assigned**

(`CREDGUARD-Validation.ps1`).

1. Force a sync (**Settings** → **Accounts** → **Access work or school** → **Info** → **Sync**).
2. Check **Devices** → **Configuration** → **(profile)** → **Device status** for an error.

Symptom: Credential Guard does not start (SecurityServicesRunning is empty)

Likely cause: A hardware prerequisite is missing/disabled, or VBS platform-security level is wrong. **Fix:**

1. Run `CREDGUARD-Discovery.ps1 -LocalReadiness` — confirm **TPM 2.0, UEFI, Secure Boot, virtualization**

(R2). Enable whatever is off in firmware.

1. In `Credential Guard - Pilot`, confirm **Turn On Virtualization Based Security = Enabled** and a valid

Select Platform Security Level (Secure Boot).

1. Reboot (VBS/Credential Guard initializes at boot) and re-check `Win32_DeviceGuard`.
2. Exclude devices that cannot meet the prerequisites from the VBS controls.

Symptom: A driver or app breaks after enabling Credential Guard / HVCI

Likely cause: An HVCI-incompatible kernel-mode driver (older VPN, endpoint agent, virtualization tool). **Fix:**

1. Review **Event Viewer** for VBS/HVCI driver-compatibility events naming the driver.
2. Update or replace the driver; keep the device in the **without-lock** pilot until clean.
3. If you must back out, unassign the profile (works while **without lock**); UEFI-locked devices require

the documented VBS-removal procedure (R9).

Symptom: LSASS protection blocks a security/SmartCard plug-in

Likely cause: An unsigned or improperly-signed LSA plug-in is incompatible with RunAsPPL. **Fix:**

1. Review **…** → **CodeIntegrity** → **Operational** (and the LSA log) for **would-block/blocked** events (e.g.

3065/3066) that name the module — this is exactly what the **audit-first** window is for (Implementation Step 7).

1. Update/remove the module, or obtain a properly-signed version.

2. Only after the audit is clean, switch `LSASS Protection (RunAsPPL) - Pilot` to **Enabled with UEFI

lock**.** Verify current event IDs in your build.**

Symptom: WDigest value still shows 1 (or missing)

Likely cause: Profile not applied, or a legacy GPO/registry is re-setting it. **Fix:**

1. Confirm `WDigest Disabled - Baseline` shows **Succeeded** and re-sync.

2. Check for a conflicting on-prem GPO or startup script setting `UseLogonCredential = 1`; remove it.

3. If the Settings Catalog does not surface the setting, deliver `UseLogonCredential = 0` via the

Custom OMA-URI / remediation fallback (Implementation Step 8).

Symptom: Outbound RDP breaks after Remote Credential Guard

Likely cause: The RDP **target host** does not support/allow Remote Credential Guard, or a mode mismatch. **Fix:**

1. Confirm the mode set in `Remote Credential Guard - Pilot` (**Require Remote Credential Guard** vs

Restrict Credential Delegation).

1. Ensure the **host** allows it — *Remote host allows delegation of non-exportable credentials*

(`AllowProtectedCreds = 1`) and NLA are **host-side** settings owned by the separate **RDP hardening** solution, not this profile.

1. For targets that cannot support it, scope those RDP flows out or relax to a compatible mode.

Symptom: Windows LAPS password is not backed up to Entra

Likely cause: Tenant feature off, unsupported build, or backup-directory mis-set. **Fix:**

1. Entra → **Devices** → **Device settings** → **Enable Microsoft Entra LAPS = Yes** (Step 11).

2. Confirm the device build supports Windows LAPS (R7) and `Windows LAPS - Baseline` sets **Backup

Directory = Azure AD (Entra) only**.

1. Check the **Microsoft-Windows-LAPS/Operational** log for the backup result and error code.

Symptom: Helpdesk cannot retrieve the LAPS password

Likely cause: The retriever lacks the read permission. **Fix:**

1. Grant a role including `microsoft.directory/deviceLocalCredentials/password/read` (R8).
2. Retrieve via Entra → **Devices** → **(device)** → **Local administrator password recovery**, or Graph.

Symptom: Works on some devices, not others

Likely cause: Mixed hardware capability or group membership. **Fix:**

1. Run `CREDGUARD-Report.ps1` to see which devices lack the applied controls.
2. Confirm capable hardware (Test 1) for the VBS controls and correct group membership (R6).
3. Re-sync stragglers; move non-capable devices out of the VBS-control scope.

Symptom: Coverage report shows fewer devices than the group

Likely cause: Some devices not synced, not enrolled, or not capable. **Fix:**

1. Run `CREDGUARD-Report.ps1` to list the gaps.
2. Run `CREDGUARD-Remediation.ps1` **(**`-WhatIf` **first)** to correct assignment/scoping.
3. Re-sync and confirm enrollment (R1) and readiness (R2).

Escalation checklist

Before opening a Microsoft support case, collect: the `CREDGUARD-Discovery.ps1` output (including `-LocalReadiness`), the profile **Device status** exports, `Win32_DeviceGuard` output, the relevant CodeIntegrity/LSA and LAPS operational-log events, the exact Windows build, the lock mode in use, and which specific control (of the six) is failing (isolates the problem to one profile).

7. Best Practices — Windows Credential Theft Protection

These practices keep the credential-theft defense precise, safe, and reversible on corporate Windows 10/11 endpoints. They assume the deployment model in the Solution Overview: six named Intune profiles deployed via **Assess** → **Configure** → **Pilot** → **Validate** → **Expand**, VBS controls on capable hardware, audit-first.

Assess hardware before you enable VBS

• **Readiness-gate Credential Guard and LSASS PPL.** Run `CREDGUARD-Discovery.ps1 -LocalReadiness` and

target only **TPM 2.0 / UEFI / Secure Boot / virtualization-capable** devices. Assigning VBS controls to unsupported hardware just generates errors.

• **Build a capable-device group.** Keep `Credential Guard - Pilot` and `LSASS Protection (RunAsPPL) -

Pilot` scoped to that group; let the low-risk controls (WDigest, WinRM, Remote Credential Guard) and LAPS go broader.

Roll out lock mode deliberately

- **Pilot without the UEFI lock; enforce with it.** Use **Enabled without lock** while proving

compatibility (easy rollback by unassigning), then **Enabled with UEFI lock** for production so malware cannot silently disable VBS.

- **Plan the sticky rollback up front.** Document that UEFI-locked VBS is removed by the documented

procedure (often console/physical access), not a profile unassign — set that expectation with change management before the production wave.

Audit LSASS protection first — always

- **Never flip RunAsPPL fleet-wide blind.** Deploy audit-first, watch **CodeIntegrity/LSA operational**

logs for would-block events (e.g. 3065/3066), remediate the named plug-in/driver, *then* enforce. This is the single biggest way to avoid a broken-SmartCard/EDR incident.

Keep the low-risk controls simple and enforced

- **WDigest and WinRM are cheap wins.** Modern Windows often already sets `UseLogonCredential = 0`, but

enforce and prove it so an audit finding is closed with evidence. Same for WinRM `DisableRunAs = 1`.

- **Restrict credential delegation on the client, harden the host separately.** This package owns the

outbound Remote Credential Guard setting; pair it with the **RDP hardening** solution for the host-side (`AllowProtectedCreds`, Restricted Admin, NLA). Don't duplicate the host settings here.

Operate Windows LAPS well

- **Use built-in Windows LAPS, not the legacy MSI**, and back up to **Entra ID only** unless you have a

hybrid requirement. Set a sane rotation (e.g. 30 days), strong length/complexity, and a post-auth reset.

- **Lock down retrieval.** Grant `deviceLocalCredentials/password/read` narrowly and log who retrieves;

the password is only as safe as the read permission.

- **Manage a named admin account where possible** rather than the built-in Administrator, so LAPS

rotation is unambiguous.

Deploy pilot-first and reversibly

- **Assign to `Credential Protection - Pilot`, never "All devices" first.** Expand in waves after each

gate passes.

• **One control at a time when debugging.** Because each control is its own named profile, you can unassign or narrow exactly the one that misbehaves without touching the others.

Operate and monitor

• **Report before you widen.** Run `CREDGUARD-Report.ps1` each wave; only broaden when coverage and runtime state match the assigned group.

• **Remediate with `-WhatIf` first.** `CREDGUARD-Remediation.ps1` and `CREDGUARD-Set-ScopeTargeting.ps1` preview by default — read the plan, then apply.

• **Watch the runtime state, not just policy status.** A profile can show "Succeeded" while Credential Guard isn't *running* on incapable hardware — validate `Win32_DeviceGuard` and the registry values.

Change control

• **Version scope changes.** When you flip a lock mode, add a device group, or change a LAPS parameter, note the date and reason; re-run Discovery and Report to confirm no regressions.

• **Review after major Windows upgrades.** Confirm Credential Guard still runs, LSASS is still protected, and LAPS still backs up after significant feature updates before expanding further.

8. Deliverables Summary — Windows Credential Theft Protection

Everything included in this solution package, and what each piece is for.

Documentation (this guide)

• **Executive Summary** — the business case in one page: how credential-theft / pass-the-hash attacks harvest secrets from LSASS and static local-admin passwords, the six controls that close those doors, and the ~3 assessment findings this resolves directly.

• **Solution Overview** — the attack chain and why each control exists; how Credential Guard (VBS/LSA isolation), LSASS protection (RunAsPPL/PPL), WDigest, Remote Credential Guard, WinRM, and Windows LAPS work; and the explicit scope boundary against RDP-host hardening and full baselines.

• **Requirements Guide** — every Intune/Entra/Windows/hardware prerequisite with how to **verify** and how to **implement if missing** (TPM/UEFI/Secure Boot/virtualization readiness, LAPS build + Entra feature, retrieval permissions, UEFI-lock rollback awareness), plus the category-level Global Requirements.

• **Implementation Guide** — exact, ordered, click-by-click steps across six phases: assess readiness, configure Credential Guard, configure LSASS protection (audit-first), configure WDigest / Remote Credential Guard / WinRM, configure Windows LAPS, then pilot / validate / expand — every object named and reused.

• **Testing & Validation** — ten tests with explicit pass/fail gates (readiness, Credential Guard running, LSASS audit-then-enforce, WDigest, Remote Credential Guard, WinRM, LAPS backup/retrieval, coverage, reversibility) and an acceptance gate.

• **Troubleshooting** — top-down diagnosis for VBS-won't-start, driver/plug-in incompatibility, WDigest persistence, RDP breakage, and LAPS backup/retrieval issues.

• **Best Practices** — readiness-gating, deliberate lock-mode rollout, audit-first LSASS, cheap-win low-risk controls, Windows LAPS operations, and reversible pilot-first deployment.

PowerShell scripts (Microsoft Graph) — `CREDGUARD-Scripts.zip`

All scripts are tenant-side (run from an admin workstation with the Microsoft Graph PowerShell SDK); each has a header explaining **Purpose · How to run · Expected results**. Discovery, Validation, and Report are **read-only**; Remediation and Set-ScopeTargeting **default to** `-WhatIf` (preview first, then apply).

1. `CREDGUARD-Discovery.ps1` — tenant readiness: the six credential-protection profiles are present and correctly configured, and (with `-LocalReadiness`) an on-device **TPM 2.0 / UEFI / Secure Boot / VBS** hardware-readiness check for Credential Guard.

1. `CREDGUARD-Validation.ps1` — validates a named profile's **assignment** and per-device **applied** state, with a clear PASS/FAIL summary.

1. `CREDGUARD-Report.ps1` — tenant-wide CSV report of Windows devices and whether the credential-protection controls are applied (plus readiness/VBS-running state where available) — the coverage report.

1. `CREDGUARD-Remediation.ps1` — safe remediation of the flagship gap: creates the **Credential Guard **endpoint-protection profile (VBS + Credential Guard, UEFI lock by default) and assigns it to a target group, then** previews **the remaining controls (LSASS RunAsPPL, WDigest, Remote Credential Guard, WinRM, LAPS) with the exact values to set per the guide.** `-WhatIf` by default**; applies only when re-run with `-WhatIf:$false`.

1. `CREDGUARD-Set-ScopeTargeting.ps1` — sets/adjusts the profile group assignment (include/exclude target-group builder) for wave-based rollout. `-WhatIf` **by default.**

Hero image

• `CREDGUARD-hero.png` — the product image used on the marketplace listing.

How it's delivered

A **Word** document and **PDF** (cover, table of contents, page numbers, and a scripts appendix), plus all five PowerShell scripts in a single **ZIP** — yours to keep and reuse across **unlimited devices**.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

CREDGUARD-Discovery.ps1

Purpose: Confirms the Intune tenant is ready to deliver the credential-theft / pass-the-hash defense pack for Windows 10/11: Windows Defender Credential Guard (VBS), LSASS protection (RunAsPPL), WDigest disabled (UseLogonCredential=0), Remote Credential Guard / restricted delegation, WinRM DisableRunAs, and Windows LAPS. It reports which of the six credential-protection profiles are present, plus whether the Entra Windows LAPS feature can be relied upon. With -LocalReadiness it also runs an ON-DEVICE hardware-readiness check for Credential Guard: TPM 2.0, UEFI firmware, Secure Boot, and virtualization-based security. This checks the ENDPOINT credential-theft controls only — not RDP host hardening, full CIS/STIG baselines, or domain-controller settings. HOW TO RUN Tenant readiness (Graph): powershell -ExecutionPolicy Bypass -File .\CREDGUARD-Discovery.ps1 Add on-device hardware readiness (run locally on a candidate device): powershell -ExecutionPolicy Bypass -File .\CREDGUARD-Discovery.ps1 -LocalReadiness (Install once: Install-Module Microsoft.Graph -Scope CurrentUser) Required Graph scopes: DeviceManagementConfiguration.Read.All EXPECTED RESULTS A PASS/FAIL line per check and an overall verdict. A READY tenant shows the six credential-protection profiles present; a READY device shows TPM 2.0, UEFI, Secure Boot, and VBS capability for Credential Guard. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph and the local device. > [CmdletBinding()] param([switch]$LocalReadiness)

How to run: Tenant readiness (Graph): powershell -ExecutionPolicy Bypass -File .\CREDGUARD-Discovery.ps1 Add on-device hardware readiness (run locally on a candidate device): powershell -ExecutionPolicy Bypass -File .\CREDGUARD-Discovery.ps1 -LocalReadiness (Install once: Install-Module Microsoft.Graph -Scope CurrentUser) Required Graph scopes: DeviceManagementConfiguration.Read.All EXPECTED RESULTS A PASS/FAIL line per check and an overall verdict. A READY tenant shows the six credential-protection profiles present; a READY device shows TPM 2.0, UEFI, Secure Boot, and VBS capability for Credential Guard. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph and the local device. > [CmdletBinding()] param([switch]$LocalReadiness)

CREDGUARD-Remediation.ps1

Purpose: Remediates the most common gap in the credential-theft defense: no Windows Defender Credential Guard profile deployed. It creates a Windows endpoint-protection profile that enables virtualization-based security and Credential Guard (UEFI lock by default) and assigns it to a target group, then PREVIEWS the remaining controls (LSASS RunAsPPL, WDigest UseLogonCredential=0, Remote Credential Guard / restricted delegation, and WinRM DisableRunAs) with the exact values to set per the Implementation Guide — because those surface differently across tenants (Settings Catalog vs Custom) and should be verified before creation. Defaults to -WhatIf, so it previews every change and modifies nothing until re-run to apply. Remediates the endpoint credential-theft controls only — never RDP host hardening or domain settings. HOW TO RUN Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\CREDGUARD-Remediation.ps1 -GroupId your-entra-group-object-id Apply the change: powershell -ExecutionPolicy Bypass -File .\CREDGUARD-Remediation.ps1 -GroupId your-entra-group-object-id -WhatIf:$false Optional: -Name "Credential Guard - Pilot" -WithoutLock (use enableWithoutUEFILock for pilots) (Install once: Install-Module Microsoft.Graph -Scope CurrentUser)

Required Graph scopes: DeviceManagementConfiguration.ReadWrite.All EXPECTED RESULTS In preview: a report of what is missing and exactly what WOULD be created/assigned, plus the values for the remaining hardening controls. When applied: a Credential Guard endpoint-protection profile exists and is assigned to the group. Re-run CREDGUARD-Discovery.ps1 to confirm. .NOTES White-label — the repeatable way. Defaults to -WhatIf; changes require -WhatIf:$false. > [CmdletBinding(SupportsShouldProcess)] param([Parameter(Mandatory)] [string]$GroupId, [string]$Name = 'Credential Guard - Pilot', [switch]$WithoutLock)

How to run: Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\CREDGUARD-Remediation.ps1 -GroupId your-entra-group-object-id Apply the change: powershell -ExecutionPolicy Bypass -File .\CREDGUARD-Remediation.ps1 -GroupId your-entra-group-object-id -WhatIf:$false Optional: -Name "Credential Guard - Pilot" -WithoutLock (use enableWithoutUEFILock for pilots) (Install once: Install-Module Microsoft.Graph -Scope CurrentUser) Required Graph scopes: DeviceManagementConfiguration.ReadWrite.All EXPECTED RESULTS In preview: a report of what is missing and exactly what WOULD be created/assigned, plus the values for the remaining hardening controls. When applied: a Credential Guard endpoint-protection profile exists and is assigned to the group. Re-run CREDGUARD-Discovery.ps1 to confirm. .NOTES White-label — the repeatable way. Defaults to -WhatIf; changes require -WhatIf:$false. > [CmdletBinding(SupportsShouldProcess)] param([Parameter(Mandatory)] [string]$GroupId, [string]$Name = 'Credential Guard - Pilot', [switch]$WithoutLock)

CREDGUARD-Report.ps1

Purpose: Produces a coverage report across managed Windows 10/11 devices: for each device, whether the named credential-theft-protection profile is applied, alongside a hardware-readiness signal (from managed-device inventory) useful for the VBS controls (Credential Guard / LSASS PPL). Use it to measure coverage before widening a rollout and to find devices still missing the credential-theft defense. Read-only. Reports the endpoint credential-protection controls only. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CREDGUARD-Report.ps1 Optional: -ProfileName "Credential Guard - Pilot" -CsvPath .\CREDGUARD-coverage.csv (Install once: Install-Module Microsoft.Graph -Scope CurrentUser) Required Graph scopes: DeviceManagementConfiguration.Read.All, DeviceManagementManagedDevices.Read.All EXPECTED RESULTS A table (and optional CSV) of Windows devices with an Applied column (Yes/No/Error) plus a coverage summary. Full coverage = every intended device shows Applied = Yes. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. > [CmdletBinding()] param([string]$ProfileName = 'Credential Guard - Pilot', [string]$CsvPath)

How to run: powershell -ExecutionPolicy Bypass -File .\CREDGUARD-Report.ps1 Optional: -ProfileName "Credential Guard - Pilot" -CsvPath .\CREDGUARD-coverage.csv (Install once: Install-Module Microsoft.Graph -Scope CurrentUser) Required Graph scopes: DeviceManagementConfiguration.Read.All, DeviceManagementManagedDevices.Read.All EXPECTED RESULTS A table (and optional CSV) of Windows devices with an Applied column (Yes/No/Error) plus a coverage summary. Full coverage = every intended device shows Applied = Yes. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. > [CmdletBinding()] param([string]$ProfileName = 'Credential Guard - Pilot', [string]$CsvPath)

CREDGUARD-Set-ScopeTargeting.ps1

Purpose: Sets or adjusts which Entra security group a Windows credential-theft-protection profile (Credential Guard, LSASS protection, WDigest, Remote Credential Guard, WinRM, or Windows LAPS) is assigned to — the target-group builder for pilot-first, wave-based rollout. Optionally adds an assignment exclusion group (e.g. to hold back non-Credential-Guard-capable hardware from the VBS controls). Defaults to -WhatIf, previewing the assignment change and modifying nothing until re-run to apply. Targets the endpoint credential-protection profiles only, not RDP host hardening or domain settings. HOW TO RUN Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\CREDGUARD-Set-ScopeTargeting.ps1 -GroupId your-entra-group-object-id Apply the assignment: powershell -ExecutionPolicy Bypass -File .\CREDGUARD-Set-ScopeTargeting.ps1 -GroupId your-entra-group-object-id -WhatIf:$false Optional: -ExcludeGroupId your-exclusion-group-object-id ; -ProfileName "Credential Guard - Pilot" (Install once: Install-Module Microsoft.Graph -Scope CurrentUser) Required Graph scopes: DeviceManagementConfiguration.ReadWrite.All EXPECTED RESULTS In preview: the profile found and the assignment that WOULD be set. When applied: the profile is assigned to the include group (and excludes the exclusion group). Confirm in Intune - Devices - Configuration, or with CREDGUARD-Validation.ps1. .NOTES White-label — the repeatable way. Defaults to -WhatIf; changes require -WhatIf:$false. > [CmdletBinding(SupportsShouldProcess)] param([Parameter(Mandatory)] [string]$GroupId, [string]$ExcludeGroupId, [string]$ProfileName = 'Credential Guard - Pilot')

How to run: Preview (default, no changes): powershell -ExecutionPolicy Bypass -File .\CREDGUARD-Set-ScopeTargeting.ps1 -GroupId your-entra-group-object-id Apply the assignment: powershell -ExecutionPolicy Bypass -File .\CREDGUARD-Set-ScopeTargeting.ps1 -GroupId your-entra-group-object-id -WhatIf:$false Optional: -ExcludeGroupId your-exclusion-group-object-id ; -ProfileName "Credential Guard - Pilot" (Install once: Install-Module Microsoft.Graph -Scope CurrentUser) Required Graph scopes: DeviceManagementConfiguration.ReadWrite.All EXPECTED RESULTS In preview: the profile found and the assignment that WOULD be set. When applied: the profile is assigned to the include group (and excludes the exclusion group). Confirm in Intune - Devices - Configuration, or with CREDGUARD-Validation.ps1. .NOTES White-label — the repeatable way. Defaults to -WhatIf; changes require -WhatIf:$false. > [CmdletBinding(SupportsShouldProcess)] param([Parameter(Mandatory)] [string]$GroupId, [string]$ExcludeGroupId, [string]$ProfileName = 'Credential Guard - Pilot')

CREDGUARD-Validation.ps1

Purpose: Validates that a Windows credential-theft-protection profile (Credential Guard, LSASS protection, WDigest, Remote Credential Guard, WinRM, or Windows LAPS) is assigned to the intended group and applied on target devices. Confirms the profile exists (Settings Catalog or template), that it is assigned, and reports each Windows device's applied state (Succeeded / Error / Not applied). Read-only. Validates the endpoint credential-protection controls only, not RDP host hardening or domain settings. HOW TO RUN powershell -ExecutionPolicy Bypass -File .\CREDGUARD-Validation.ps1 -ProfileName "Credential Guard - Pilot" Optional: -GroupId your-entra-group-object-id (assert the profile is assigned to this group) (Install once: Install-Module Microsoft.Graph -Scope CurrentUser) Required Graph scopes: DeviceManagementConfiguration.Read.All EXPECTED RESULTS A PASS/FAIL summary: profile found, assigned (to -GroupId if supplied), and a per-device applied-state table. PASS when the profile is assigned and pilot devices report Succeeded. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. > [CmdletBinding()] param([string]$ProfileName = 'Credential Guard - Pilot', [string]$GroupId)

How to run: powershell -ExecutionPolicy Bypass -File .\CREDGUARD-Validation.ps1 -ProfileName "Credential Guard - Pilot" Optional: -GroupId your-entra-group-object-id (assert the profile is assigned to this group) (Install once: Install-Module Microsoft.Graph -Scope CurrentUser) Required Graph scopes:

DeviceManagementConfiguration.Read.All EXPECTED RESULTS A PASS/FAIL summary: profile found, assigned (to -GroupId if supplied), and a per-device applied-state table. PASS when the profile is assigned and pilot devices report Succeeded. .NOTES White-label — the repeatable way. Read-only against Microsoft Graph. > [CmdletBinding()] param([string]$ProfileName = 'Credential Guard - Pilot', [string]$GroupId)